

C++ | CPl

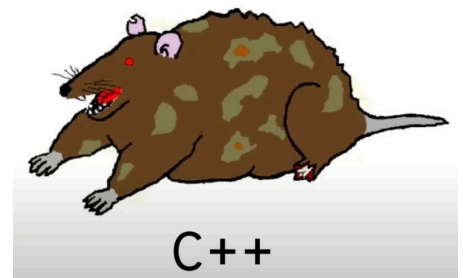
Summary

CONTENTS

1. Why C++?	2
2. Intro	2
2.1. Compilation	2
2.2. Declarations and definitions	3
2.2.1. #include guards	3
2.2.2. Functions	3
2.2.3. Values and references	3
2.3. Libraries	4
3. Values and Streams	4
3.1. Variables	4
Bibliography	5

1. WHY C++?

- Multi-paradigm language, now you have a thousand and one way to shoot yourself in your foot and obliterate your PC
- Being called stupid when you think that the move function actually moves things or if you confuse a const pointer with a pointer of const
- ISO standard: C++23. The higher the number the hotter the mess of poorly designed choices that have accumulated over decades into a chaotic pile of obsolescence
- Good excuse for alcoholism
- Zero-cost abstractions (if you like learning the whole stdlib by heart and don't mind the cost of debugging template errors for hours on end)
- Headaches (if you're a masochist)
-
- It's a never ending journey. Even if you dedicate your whole life to learning C++, you'll never reach the day where you'll know all of the stdlib.
- "Undefined behavior" is defined (in the standard)
- Ever wanted to have 100 different ways to do the same thing? Now you can
- Has a pretty mascot [\[1\]](#)



2. INTRO

- Implicitly returns 0
- Return types are written in front of the function name or as trailing return-types in declarations (because who needs consistency, amirite)

Nice and clean: `int main() { }` Insanity: `auto main() → int { }`

- Header files can be either `*.hpp` or `*.h`
- Regarding include order: It is good practice to include own-headers ("`"`") before system-headers ("`<`"). The first is usually used for the header files belonging to the project itself, while the latter is used for includes of the standard library and external libraries. However, ***this difference is only conventional, as any toolchain could just specify their own lookup rules for the two distinct kinds of includes.***

Do's	Don'ts
	Using global variables
	Redundant returns that would be implicit
	<code>using namespace</code>
	Not initializing a variable upon definition
	Not using <code>const</code> enough
	TODO: make this column real big for the lulz

2.1. COMPILATION

3 Phases of compilation:

Preprocessor: Textual replacement of preprocessor directives (`#include`)

Compiler: Translation of C++ code into machine code (source file to object file)

Linker: Combination of object files and libraries into libraries and executables

2.2. DECLARATIONS AND DEFINITIONS

All things with a name that you use in a C++ program must be declared before you can do so

- E.g. a function that you call (major difference from C)
- A type that you use for a variable (except some built-ins)
- A variable that you use

There can only be one definition of the same function (**One Definition Rule**/ODR), in contrast to declaration, which can be done several times.

2.2.1. `#include` guards

`#include` guards ensure that a header file is only included once. Necessary for functions and class type definitions, good practice either way.

```
#ifndef GUARD_HPP_
#define GUARD_HPP_
// ...
#endif /* GUARD_HPP_ */
```

2.2.2. Functions

Declarations

```
auto <function-name>(<parameters>) → <return-type>;
<return-type> <function-name>(<parameters>);
```

Definitions

```
auto <function-name>(<parameters>) → <return-type> { /*body*/ }
<return-type> <function-name>(<parameters>) { /*body*/ }
```

If a function has a non-void return type it must return a value on every path (or throw an exception).

2.2.3. Values and references

```
struct Point {
    int x;
    int y;
};
```

Copying vs. Sharing:

```
Point point{1, 20};
```

```
Point copiedPoint{point};
```

```
Point &sharedPoint{point};
```

```
auto copydPointParam(Point point) → Point { }
auto referencePointParam(Point & point) → Point { }
```

2.3. LIBRARIES

- [catch2](#)
- [boost](#)

TODO:

cmake

3. VALUES AND STREAMS

3.1. VARIABLES

```
<type> <variable-name>{<initial-value>;
```

Using = or {} for initialization with a value supplied we can have the compiler determine its type.

```
auto const i = 5;
```

Initialization might be omitted but that is bad practice and potentially dangerous.

```
double x;
```

Adding the const keyword in front of the name makes the variable a single-assignment variable, aka a constant which is immutable and must be initialized. Some constants are required to be fixed at compile time (**constexpr**). The keyword **const** also appears in other contexts.

```
int const theAnswer{42};  
double constexpr pi{3.14};
```

You should use **const** whenever possible for non-member variables.

BIBLIOGRAPHY

- [1] "C++ Mascot." Accessed: Sept. 12, 2026. [Online]. Available: https://media.licdn.com/dms/image/v2/D4D22AQFwZZ5wmpZmnw/feedshare-shrink_800/feedshare-shrink_800/0/1724256362408?e=2147483647&v=beta&t=OXxfC_GlaJFqlUgO03e-9B59Zpzu4-LYG90IM8k4UFE